클래식 음악 퀴즈 게임 과제 보고서

입학연수 · 개발 입문 | 조민경 | 2026. 11. 08.

1. 과제 개요

파이썬 표준 라이브러리만으로 클래식 음악을 주제로 한 터미널 퀴즈 게임을 만들고, 개발 과정 전체를 Git과 GitHub로 관리했습니다.

항목	내용
과제명	컴퓨터에게 명령 내리는 말(파이썬) 처음 배우기
과정 구분	입학연수 · 개발 입문 (필수, 학습시간 40시간)
결과물	클래식 음악 퀴즈 게임 (터미널 실행)
저장소 URL	https://github.com/coolgirl2046/classical-music-quiz
운영체제	Windows
Python	3.13.15 (외부 라이브러리 미사용)
Git	2.56.0.windows.2
편집기	Visual Studio Code

과제 목표와 적용 내용

과제가 제시한 4가지 학습 목표를 이번 프로젝트의 어느 부분에서 직접 사용했는지 정리했습니다.

학습 목표	프로젝트에서 적용한 부분
Python 기초 (변수, 자료형, 조건문, 반복문, 함수)	퀴즈 목록은 list, 저장 데이터는 dict, 최고 점수는 int, 문제 글은 str, 정답 확인 결과(True/False)는 bool로 관리. 메뉴 분기는 if/elif/else, 문제 출제는 for, 올바른 입력이 나올 때까지 반복하는 부분은 while 사용. read_number(prompt, min, max)처럼 매개변수와 반환값이 있는 함수 작성
클래스와 객체	Quiz, QuizGame 클래스 정의. __init__에서 self로 속성을 만들고, 기능별 메서드로 역할 분리
파일 입출력	with open(..., encoding="utf-8")과 json 모듈로 state.json 저장·불러오기. try/except로 숫자 변환 실패, 파일 손상, Ctrl+C 처리
Git 기초	init, add, commit, push, pull, checkout, clone 사용. 기능 브랜치 2개 생성 후 병합, 복제본에서 올린 변경을 pull로 가져옴

2. 주제 선정 이유

피아노를 전공했고, 아이들에게 작곡가 퀴즈 수업을 했던 경험이 있습니다. 그 수업을 떠올리며, 익숙하고 애정이 있는 클래식 음악을 퀴즈 주제로 정했습니다.

기본 문제 5개는 베토벤, 비발디, 바흐, 모차르트, 차이코프스키 등 널리 알려진 작곡가와 작품으로 구성했습니다.

3. 구현 기능과 프로그램 구조

메뉴 5개를 모두 구현했고, 보너스 과제 중 랜덤 출제 기능을 추가했습니다.

메뉴	기능	담당 메서드
1. 퀴즈 풀기	저장된 퀴즈를 무작위 순서로 출제하고 점수(100점 만점)를 계산	play_quiz
2. 퀴즈 추가	문제, 선택지 4개, 정답 번호를 입력받아 추가 후 즉시 저장	add_quiz
3. 퀴즈 목록	등록된 퀴즈의 문제를 번호와 함께 출력	show_quiz_list
4. 점수 확인	최고 점수 출력, 기록이 없으면 안내 문구 출력	show_best_score
5. 종료	데이터를 저장하고 프로그램 종료	save_state

클래스 설계

- **Quiz** (quiz.py): 퀴즈 한 문제를 표현합니다. 속성은 question, choices(4개), answer(1~4)입니다. 문제 출력(display), 정답 확인(is_correct), 딕셔너리 변환(to_dict, from_dict) 메서드를 가집니다.
- **QuizGame** (main.py): 게임 전체를 관리합니다. 퀴즈 목록(quizzes)과 최고 점수(best_score)를 갖고, 메뉴 출력, 입력 처리, 각 기능, 파일 저장·불러오기를 기능별 메서드로 나눠 처리합니다.

파일 구조

파일 또는 경로	역할
classical-music-quiz/	프로젝트 최상위 폴더
main.py	QuizGame 클래스와 프로그램 시작점
quiz.py	Quiz 클래스
README.md	프로젝트 설명
.gitignore	Git에 올리지 않을 파일 목록
docs/screenshots/	실행 화면 캡처 이미지
state.json	실행 중 자동 생성 (Git 제외)

데이터 파일 (state.json)

프로젝트 폴더에 UTF-8로 저장되며, 프로그램을 다시 실행해도 추가한 퀴즈와 최고 점수가 유지됩니다.

키	형식	설명
quizzes	리스트	퀴즈 목록. 각 항목은 question, choices, answer를 가짐
best_score	숫자 또는 null	최고 점수. 한 번도 풀지 않았으면 null

4. 입력·예외 처리

잘못된 입력이나 파일 문제가 있어도 프로그램이 멈추지 않도록 처리했습니다.

상황	처리 방식
입력 앞뒤 공백	strip()으로 제거
빈 입력, 숫자가 아닌 입력, 범위 밖 숫자	안내 문구 출력 후 다시 입력받음 (read_number)
Ctrl+C 또는 입력 종료(EOF)	데이터를 저장하고 안전하게 종료. 저장에 실패하면 "저장되지 못했습니다"로 구분해 안내
state.json 파일이 없음	기본 퀴즈 5개로 시작
state.json 파일이 손상됨	안내 문구 출력 후 기본 퀴즈로 복구하고 다시 저장. 정답이 정수 1~4가 아니거나(예: 2.0), 문제·선택지가 빈 글자인 경우도 손상으로 판단
등록된 퀴즈가 없음	풀기·목록 메뉴에서 안내 문구 출력

파일 저장과 불러오기, 손상 파일 복구는 직접 실행해 동작을 확인했습니다.

5. Git 작업 내역

커밋 21개(병합 커밋 2개 포함)를 기능 단위로 만들었고, 브랜치 2개를 만들어 main에 병합했습니다. 병합은 `--no-ff` 옵션을 사용해 그래프에 브랜치 흐름이 남도록 했습니다.

커밋 기록 (최신순)

커밋	메시지	브랜치
dba8f5b	Docs: README 예외 처리 설명을 수정된 동작에 맞게 갱신	main
aabfc55	Docs: Git 그래프 스크린샷 최신화	main
c2cf8cd	Fix: 저장 실패 시 종료 안내 문구를 실제 저장 결과에 맞게 구분	main
bbbf775	Fix: 저장 파일의 정답 번호(정수).문제,선택지 형식 검사 강화	main
d90dbc9	Docs: 비전공자도 이해할 수 있도록 코드에 쉬운 주석 추가	main
7459119	Docs: README에 실행 화면 스크린샷 추가	main
0d921ee	Docs: README에 clone 실습 기록 추가	main
0b8c09e	Docs: README 작성 (개요, 선정이유, 실행 방법, 기능, 파일 구조, 데이터 설명)	main
3978db9	Merge: 보너 기능스(랜덤 출제) 병합	main
dd7c7be	Feat: 퀴즈 랜덤 출제 기능 추가 (보너스)	feature/bonus
91eca71	Refactor: Quiz 클래스를 quiz.py로 분리	main
c6918a2	Feat: state.jason 저장 및 불러오기 기능 구현 (손상 파일 복구 포함)	main
fcca373	Feat: 최고 점수 확인 및 갱신 기능 구현	main
0451a20	Feat: 퀴즈 목록 보기 기능 구현	main
f1a80fa	Feat: 퀴즈 추가 기능 구현 (입력 검증 포함)	main
f05eb31	Merge: 퀴즈 풀기 기능 브랜치 병합	main
71be331	Feat: 퀴즈 풀기 기능 구현 (출제, 정답 판정, 결과 표시)	feature/play-quiz
bc5e408	Feat: 클래식 음악 기본 퀴즈 데이터 5개 추가	main
d1e5723	Feat: Quiz 클래스 구현 (문제 출력, 정답확인)	main
7d4d74e	Feat: 메뉴 출력 및 입력 예외 처리 기능 구현	main
c516ccc	Chore: 프로젝트 초기 설정 (.gitignore, README 추가)	main

커밋 메시지 중 "보너 기능스"(보너스 기능)와 "state.jason"(state.json)은 오타입니다. 이미 원격 저장소에 올라간 기록을 다시 쓰면 이력이 꼬일 수 있어 그대로 두었습니다.

맨 위 두 커밋(dba8f5b, aabfc55)은 Git 그래프 스크린샷(6-6)을 찍은 뒤에 올린 것이라 6-6 그래프에는 보이지 않습니다. 하나는 README의 예외 처리 설명을 고친 커밋이고, 다른 하나는 그래프 캡처 파일을 올린 커밋입니다. 그 아래 Fix 커밋 2개는 평가 피드백을 반영한 수정입니다(7장 참고). 스크린샷 이미지는 docs/screenshots/ 폴더에 있습니다.

Git 명령어 사용 내역

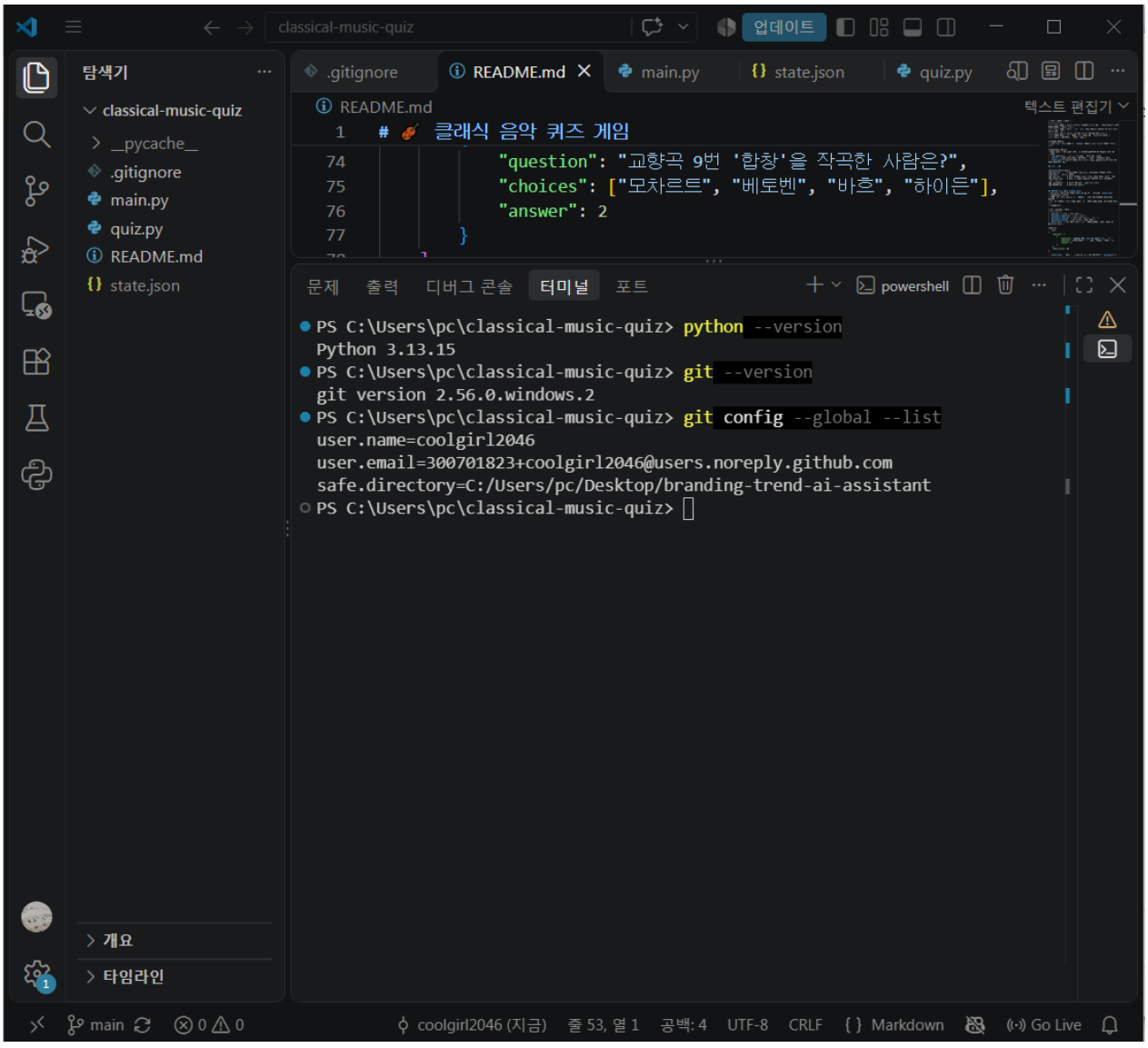
명령어	사용한 상황
git init	프로젝트 폴더를 Git 저장소로 만듦
git add	변경한 파일을 커밋 대상에 올림
git commit	기능을 하나 완성할 때마다 기록
git push	로컬 커밋을 GitHub에 업로드
git checkout	기능 브랜치 생성(-b)과 main 복귀
git clone	저장소를 다른 폴더(quiz-clone)에 복제해 README 수정 후 push
git pull	복제본에서 올린 변경을 원래 폴더로 내려받음
git merge	기능 브랜치를 main에 병합

6. 실행 결과 스크린샷

개발 환경, 메뉴별 실행 화면 4개, Git 그래프를 캡처했습니다.

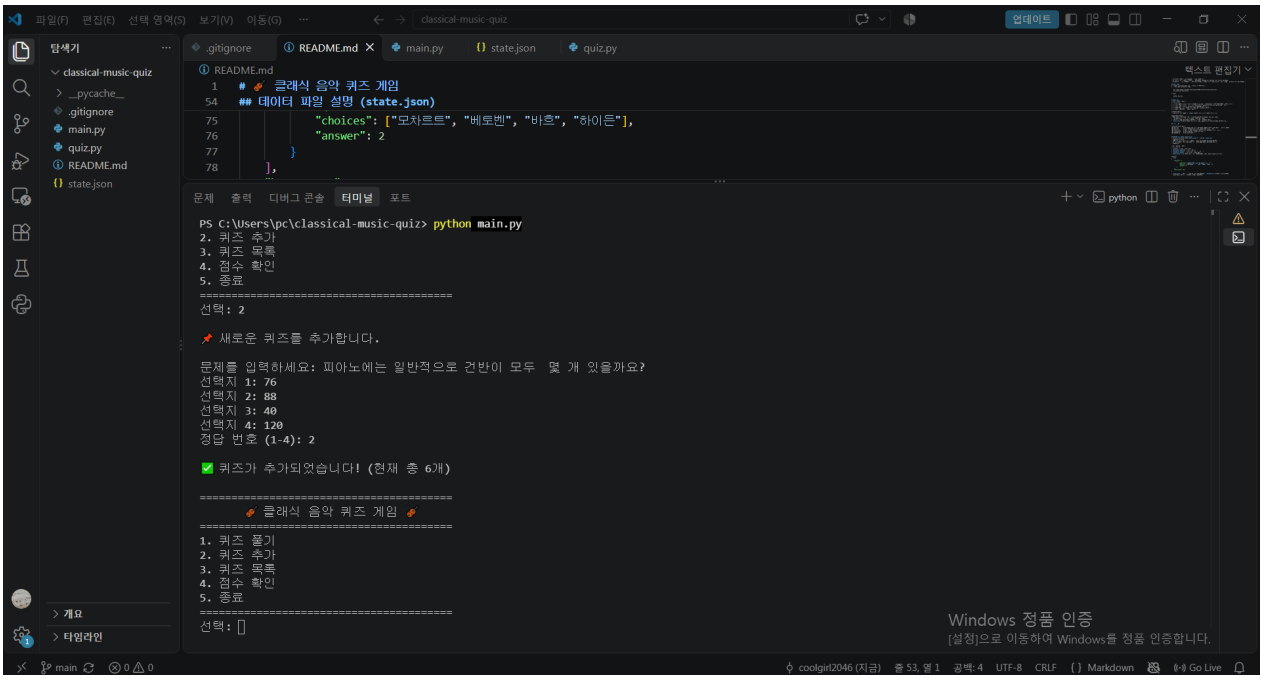
6-1. 개발 환경

Python, Git 버전과 Git 사용자 설정을 확인한 화면입니다.



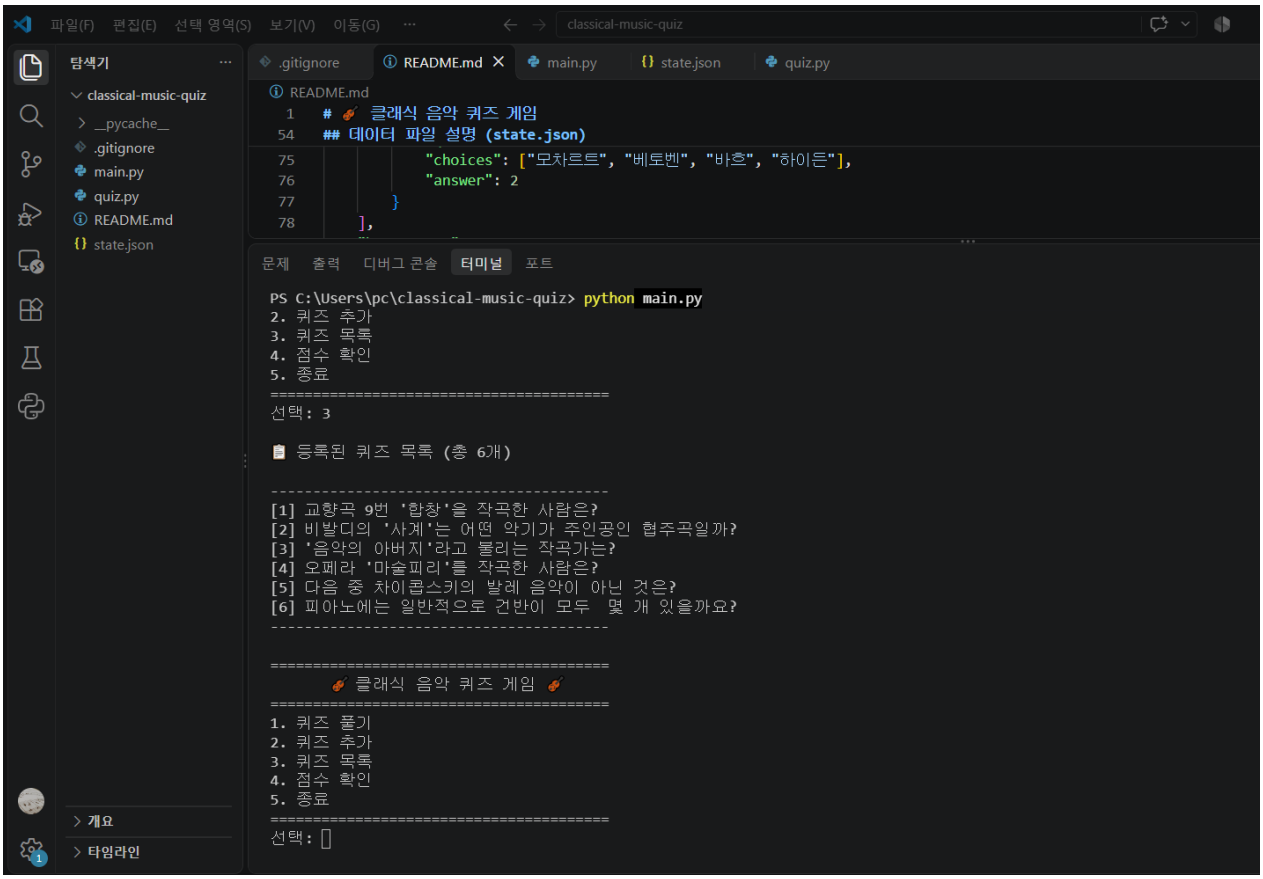
6-2. 퀴즈 추가

"피아노에는 일반적으로 건반이 모두 몇 개 있을까요?" 문제를 추가해 퀴즈가 총 6개가 되었습니다.



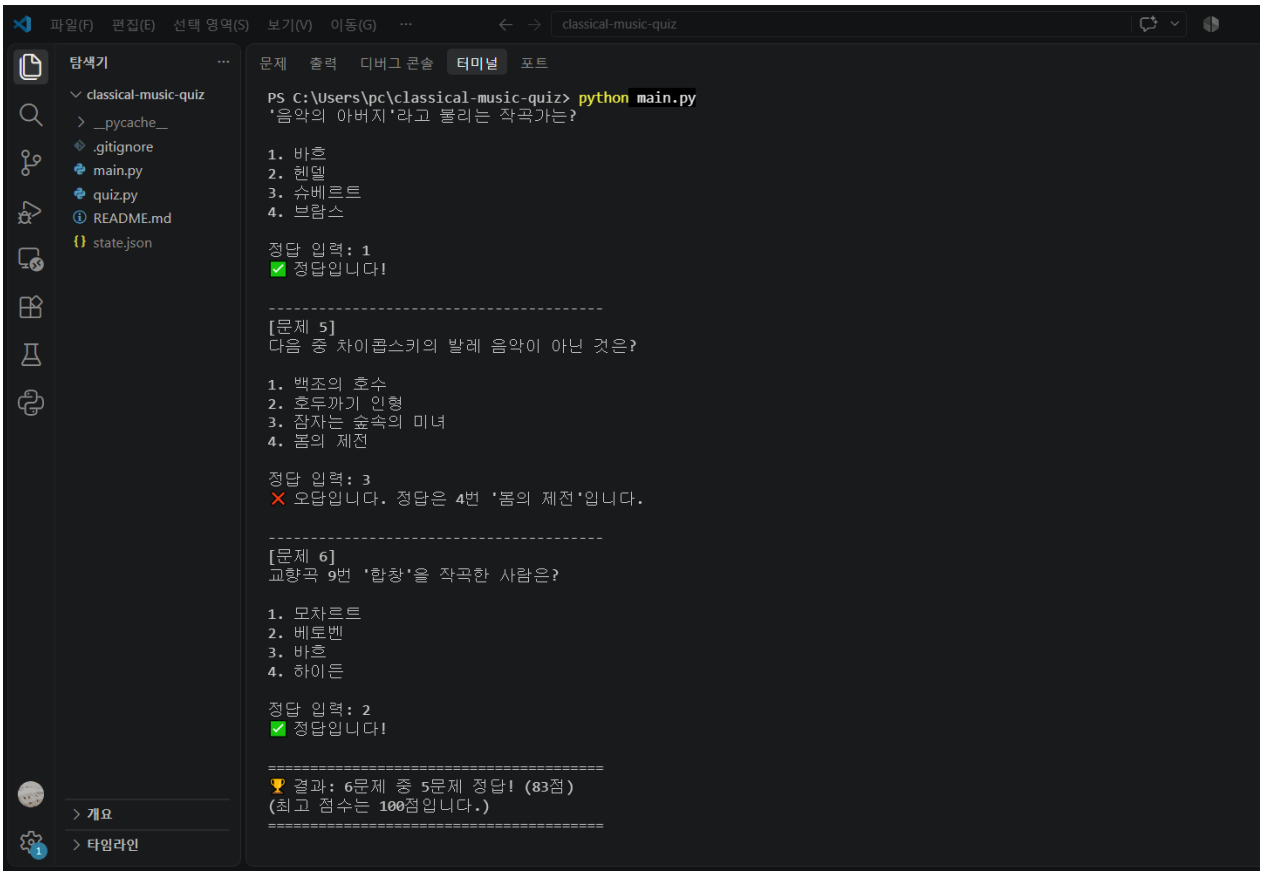
6-3. 퀴즈 목록

추가한 문제를 포함해 6개의 퀴즈가 출력됩니다.



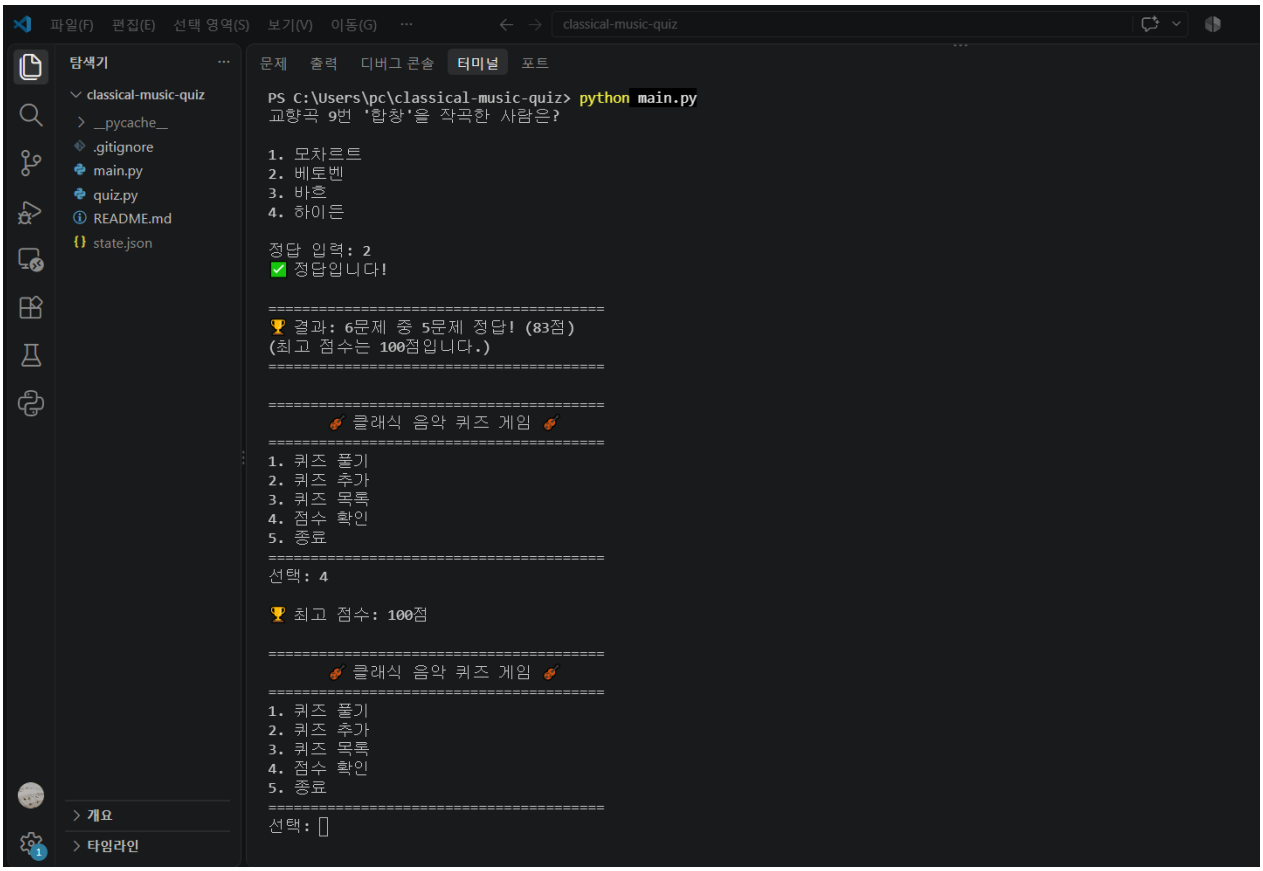
6-4. 퀴즈 풀기

문제는 무작위 순서로 출제되고, 오답이면 정답을 알려줍니다. 6문제 중 5개를 맞혀 83점을 받았습니다.



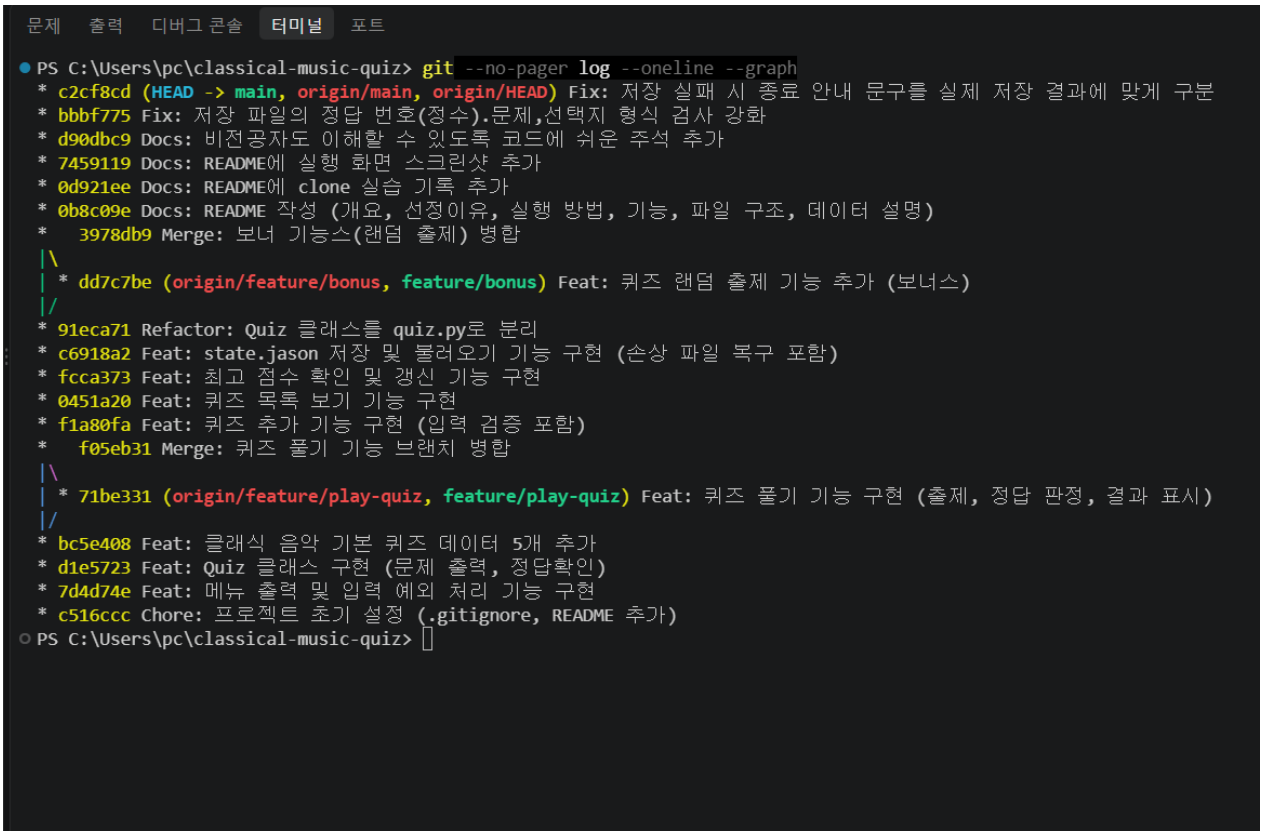
6-5. 점수 확인

저장된 최고 점수 100점이 출력됩니다. 이번 점수(83점)가 더 낮아 최고 점수는 그대로 유지되었습니다.



6-6. Git 커밋 그래프

`git log --oneline --graph` 결과로, 두 기능 브랜치가 갈라졌다가 main에 병합된 흐름이 보입니다.



7. 어려웠던 점과 배운 점

저는 비전공자여서 과제를 해 나가는 과정이 쉽지 않았습니다. 그래서 교육과정의 학습 AI인 네이토 대신 AI 도구인 Claude의 도움을 받아 진행했습니다. 전체 코드를 모두 손으로 입력하지는 못했지만, 최소한의 코드는 직접 입력하며 따라갔고 그 과정에서 오류를 많이 만났습니다. 아래는 그중 기억에 남는 오류와 배운 점입니다.

- **GitHub 이메일 거부(GH007):** 개인 이메일로 커밋하자 push가 거부되었습니다. 커밋에는 작성자 이메일이 함께 기록되어 누구나 볼 수 있는데, 제 계정은 이메일을 숨기도록 설정되어 있었기 때문입니다. 그래서 GitHub이 제공하는 noreply 이메일로 바꾸고 커밋을 수정해 해결했습니다. noreply 이메일은 GitHub이 대신 만들어 주는 가림용 주소로, 택배를 받을 때 집 주소 대신 편의점 택배 주소를 쓰는 것과 비슷합니다. 이름의 noreply는 '답장 받지 않음'이라는 뜻으로, 실제로 메일을 받는 주소는 아닙니다.
- **파일 분리(Refactor) 중 오류:** Quiz 클래스를 main.py에서 quiz.py로 옮길 때, 복사·잘라내기를 하다가 '저장된 문제를 불러오는 기능(from_dict)'이 quiz.py로 옮겨지지 않았습니다. 실행하자 "Quiz에는 from_dict라는 기능이 없다"는 오류(AttributeError)가 났습니다. 이사하면서 상자 하나를 옛날 집에 두고 온 것과 같은 상황이었습니다. quiz.py를 다시 정리하고 main.py에 남은 중복 코드를 지워 해결했습니다. 오류 메시지를 잘 읽으면 무엇이 빠졌는지 알 수 있다는 것을 배웠습니다.
- **새 기능을 메뉴에 연결하기:** 메서드를 만들고 `run()`에 연결하지 않아 메뉴를 눌러도 반응이 없던 경험이 있습니다. 메서드는 만드는 것과 호출하는 것이 따로라는 점을 알게 되었습니다. 메서드와 run()의 뜻은 부록 C의 보충 설명에 쉽게 정리해 두었습니다.
- **브랜치와 병합:** 기능을 별도 브랜치에서 만든 뒤 main에 합치는 흐름을 직접 해 보며, 그래프로 작업 이력을 한눈에 볼 수 있다는 점을 배웠습니다.
- **정답 번호 2.0 문제 (피드백 반영):** 저장 파일의 정답이 `2.0`이면 파이썬은 `2`와 같다고 보기 때문에 정상 문제로 불러왔고, 오답을 입력하자 정답 보기를 꺼내는 과정에서 TypeError가 나 게임이 꺼졌습니다. 정답이 "진짜 정수"인지(`type(answer) is int`) 확인하도록 고쳐, 이제는 손상된 파일로 판단하고 기본 퀴즈로 복구합니다. 값이 같아 보여도 자료형은 다를 수 있다는 점을 배웠습니다. 같은 피드백에서, 저장에 실패했는데도 "저장하고 종료합니다"라고 안내하던 문구를 실제 저장 결과에 맞게 나누었습니다.

앞으로는 커밋 전에 메시지와 설정을 한 번 더 확인하는 습관을 들이려고 합니다.

8. 요구사항 충족 점검표

필수 요구사항을 모두 충족했고, 보너스 중 랜덤 출제를 구현했습니다.

요구사항	충족 여부	근거
Python 3.10 이상, 외부 라이브러리 미사용	충족	Python 3.13.15, json·os·random만 사용
메뉴 5개 (풀기, 추가, 목록, 점수, 종료)	충족	3장, 6장
주제 퀴즈 5개 이상	충족	클래식 음악 기본 퀴즈 5개
클래스 2개 이상, 기능별 메서드 분리	충족	Quiz, QuizGame
state.json 저장 (UTF-8, quizzes·best_score)	충족	재실행 후 데이터 유지 확인
입력 예외 처리 (공백, 빈 입력, 문자, 범위)	충족	4장
Ctrl+C·EOF 안전 종료, 파일 없음·손상 시 복구	충족	4장
의미 있는 커밋 10개 이상	충족	커밋 21개
브랜치 생성과 병합 1회 이상	충족	feature/play-quiz, feature/bonus
init, add, commit, push, pull, checkout, clone 사용	충족	5장
.gitignore 작성	충족	`__pycache__/`, state.json 등 제외
README 6개 항목	충족	개요, 선정 이유, 실행 방법, 기능, 파일 구조, state.json
제출물 (저장소 URL, 환경·실행·Git 그래프 스크린샷)	충족	1장 저장소 URL, 6장 스크린샷 6장
보너스: 랜덤 출제	구현	`random.shuffle` 사용

9. 마무리

비전공자인 저에게 이번 과제는 끝이 아니라 시작입니다. 과제를 마쳤다고 해서 모든 내용을 완전히 이해한 것은 아니지만, 오류를 하나씩 해결하며 배우는 방법을 알게 되었습니다.

아래 부록은 발표 준비를 위한 자료입니다. 부록 A는 발표 순서, 부록 B는 비전공자도 이해할 수 있는 개념 설명, 부록 C는 예상 질문과 답변입니다.

부록 A. 발표 순서 (시연 흐름)

발표는 "무엇을 만들었는지 → 실제 동작 → 어떻게 만들었는지 → 어떻게 기록했는지" 순서로 진행합니다.

1. **소개:** 주제와 선정 이유를 말합니다. (2장)
2. **실행 시연:** `python main.py`를 실행해 메뉴를 보여 줍니다.
    a. 퀴즈 추가(2) → 목록(3) → 풀기(1) → 점수 확인(4) 순서로 시연합니다.
    b. 메뉴에서 `abc`, `9`, 빈 Enter를 입력해 예외 처리를 보여 줍니다.
    c. 종료(5) 후 다시 실행해 "저장된 데이터를 불러왔습니다" 문구로 데이터 유지를 보여 줍니다.
3. **코드 구조:** `quiz.py`의 Quiz 클래스, `main.py`의 QuizGame 클래스를 열어 역할을 설명합니다. (3장)
4. **데이터 저장:** `state.json` 파일을 열어 quizzes와 best_score를 보여 줍니다.
5. **Git 기록:** `git --no-pager log --oneline --graph`를 실행해 브랜치가 갈라졌다 합쳐진 부분을 설명합니다. (5장)
6. **마무리:** 어려웠던 점과 배운 점을 말합니다. (7장)

부록 B. 개념 쉽게 이해하기

프로그래밍을 처음 접하는 분도 따라올 수 있도록, 먼저 한 줄 비유로 전체를 보고 이어서 개념별로 자세히 설명합니다.

비전공자를 위한 한 줄 비유

개념을 처음 듣는 분들께는 아래 비유로 먼저 설명하고, 이어서 코드 예시를 보여 드립니다.

개념	쉽게 말하면	우리 프로그램에서는
변수	이름표를 붙인 상자	best_score 상자에 최고 점수를 넣어 둠
list	순서대로 줄 서 있는 기차 칸	보기 4개, 퀴즈 목록
dict	"이름: 내용"이 적힌 명함첩	"question": "문제 글"
if / elif / else	갈림길 표지판	1번을 고르면 풀기, 2번이면 추가
for	출석부를 처음부터 끝까지 한 명씩 부르기	문제를 하나씩 출제
while	"맞게 말할 때까지 다시!"	올바른 숫자를 입력할 때까지 다시 묻기
함수	자판기: 돈(매개변수)을 넣으면 음료(반환값)가 나옴	read_number에 범위를 넣으면 올바른 숫자가 나옴
클래스 / 객체	붕어빵 틀 / 붕어빵	Quiz 틀로 문제 5개를 찍어 냄
__init__ / self	붕어빵에 팥을 넣는 첫 단계 / "나 자신"	문제·보기·정답을 처음에 넣어 둠
속성 / 메서드	가진 것 / 할 수 있는 일	문제 글을 가짐 / 문제를 보여주고 채점함
파일 저장(JSON)	게임을 끄기 전에 공책에 적어 두기	state.json 공책에 문제와 점수 기록
try / except	"일단 해 보고, 넘어지면 이렇게 해!"라는 안전망	공책이 찢어져도 기본 문제로 다시 시작
Git / commit	게임의 세이브 포인트	기능을 하나 만들 때마다 저장 (21번)
push / pull	클라우드에 올리기 / 내려받기	GitHub에 올리고 받기
branch / merge	연습장을 따로 만들어 연습한 뒤, 완성되면 본무대에 합치기	풀기 기능과 랜덤 기능을 따로 만들고 합침
clone	저장소 통째로 복사본 만들기	quiz-clone 폴더에 복사

개념 자세히 보기

질문을 받았을 때 "뜻 → 비유 → 우리 코드" 순서로 설명하면 됩니다.

1) 변수

- **뜻:** 값을 저장해 두고 이름으로 다시 꺼내 쓰는 것입니다. 값은 나중에 바뀔 수 있습니다.
- **비유:** 이름표를 붙인 상자입니다. 상자 안 내용물은 바꿀 수 있어도 이름표는 그대로입니다.
- **왜 쓰나:** 같은 값을 여러 곳에서 쓰고, 바뀐 값을 기억하기 위해서입니다.
- **우리 코드:** 맞힌 개수를 세는 상자는 0에서 시작해 정답일 때마다 1씩 늘어납니다.

```
correct_count = 0      # 맞힌 개수 상자, 0으로 시작

correct_count += 1     # 정답이면 1 늘리기
```

2) list (리스트)

- **뜻:** 여러 값을 순서대로 담는 상자입니다. 대괄호 [ ]로 만듭니다.
- **비유:** 번호가 붙은 기차 칸입니다. 칸 번호는 0부터 시작합니다.
- **우리 코드:** 보기 4개와 퀴즈 목록이 list입니다. 정답 번호는 1부터 세고 list는 0부터 세기 때문에, 정답 보기를 꺼낼 때 1을 뺍니다.

```
["모차르트", "베토벤", "바흐", "하이든"]   # 0번~3번 칸

right = quiz.choices[quiz.answer - 1]   # 정답 2번 → 1번 칸 = "베토벤"
```

3) dict (딕셔너리)

- **뜻:** "이름표(키): 내용(값)" 짝으로 담는 상자입니다. 중괄호 { }로 만듭니다.
- **비유:** 명함첩입니다. 번호가 아니라 이름으로 찾습니다.
- **list와 차이:** list는 "몇 번째"로, dict는 "이름"으로 꺼냅니다.
- **우리 코드:** 문제 하나를 파일에 저장할 때 dict로 바꿉니다.

```
{"question": "교향곡 9번 '합창'을 작곡한 사람은?", "choices": [...], "answer": 2}
```

4) if / elif / else

- **뜻:** 조건에 따라 다른 일을 하게 합니다. if는 "만약", elif는 "아니면 만약", else는 "나머지 전부"입니다.
- **비유:** 갈림길 표지판입니다. 위에서부터 하나씩 확인하고, 맞는 길 하나로만 갑니다.
- **우리 코드:** 메뉴 번호에 따라 다른 기능을 실행합니다.

```
if choice == 1:
  self.play_quiz()
elif choice == 2:
  self.add_quiz()
else:
  pass  # 이 예시에서는 3~5번 메뉴 처리를 생략합니다.
```

5) for

- **뜻:** 정해진 대상을 처음부터 끝까지 하나씩 꺼내며 반복합니다.
- **비유:** 출석부를 처음부터 끝까지 한 명씩 부르기. 몇 번 반복할지 미리 알고 있습니다.
- **우리 코드:** 섞은 문제를 하나씩 출제하고, 보기 4개를 하나씩 입력받습니다.

```
for number, quiz in enumerate(questions, start=1):
  quiz.display(number)   # 문제를 하나씩 보여주기
```

6) while

- **뜻:** 조건이 맞는 동안 계속 반복합니다. 몇 번 반복할지 미리 모를 때 씁니다.
- **비유:** "맞게 말할 때까지 다시!" 답이 한 번에 맞을지 열 번 만에 맞을지 모릅니다.
- **for와 차이:** for는 횟수가 정해져 있고, while은 조건이 끝낼 때까지입니다.
- **우리 코드:** 올바른 숫자가 나올 때까지 다시 묻고, 종료를 고를 때까지 메뉴를 반복합니다. break를 만나면 반복을 끝냅니다.

```
while True:
  choice = self.read_number("선택: ", 1, 5)
  if choice == 5:
    break  # 종료를 고르면 반복을 끝냅니다.
```

7) 함수 (매개변수와 반환값)

- **뜻:** 자주 쓰는 일을 묶어 이름을 붙인 것입니다. 매개변수는 넣는 값, 반환값은 돌려받는 값입니다.
- **비유:** 자판기입니다. 돈과 버튼(매개변수)을 넣으면 음료(반환값)가 나옵니다.
- **우리 코드:** read_number에 안내 문구와 범위를 넣으면 검사를 통과한 숫자가 나옵니다. 한 번 만들어 메뉴(1~5)와 정답(1~4)에 다시 씁니다.

```
choice = self.read_number("선택: ", 1, 5)        # 메뉴용
user_answer = self.read_number("정답 입력: ", 1, 4)  # 정답용
```

8) 클래스 / 객체

- **뜻:** 클래스는 설계도, 객체는 그 설계도로 만든 실제 물건입니다.
- **비유:** 붕어빵 틀과 붕어빵. 틀은 하나지만 팥 대신 슈크림을 넣는 식으로 내용이 다른 붕어빵을 여러 개 만들 수 있습니다.
- **왜 쓰나:** 문제 하나에 필요한 것(문제·보기·정답)과 할 일(보여주기·채점)을 한 묶음으로 정리하기 위해서입니다.
- **우리 코드:** Quiz 틀 하나로 내용이 다른 문제 객체를 만듭니다.

```
Quiz("교향곡 9번 '합창'을 작곡한 사람은?", ["모차르트", "베토벤", "바흐", "하이든"], 2)
Quiz("'음악의 아버지'라고 불리는 작곡가는?", ["바흐", "헨델", "슈베르트", "브람스"], 1)
```

9) `__init__` / `self`

- **뜻:** __init__은 객체가 만들어질 때 자동으로 한 번 실행되는 준비 단계입니다. self는 지금 만들어지고 있는 "바로 그 객체 자신"입니다.
- **비유:** 붕어빵을 찍을 때 제일 먼저 틀에 반죽과 소를 넣는 단계가 __init__, "이 붕어빵의 소"라고 가리킬 때의 "이"가 self입니다.
- **우리 코드:** 문제를 만들 때 받은 값을 그 문제 자신에게 넣어 둡니다.

```
def __init__(self, question, choices, answer):
  self.question = question   # 이 문제의 문제 글
  self.choices = choices     # 보기 목록
  self.answer = answer       # 이 문제의 정답
```

10) 속성 / 메서드

- **뜻:** 속성은 객체가 가진 정보, 메서드는 객체가 할 수 있는 일입니다.
- **비유:** 사람으로 치면 이름·나이는 속성, 걷기·말하기는 메서드입니다.
- **우리 코드:** Quiz의 속성은 question, choices, answer, 메서드는 display()(보여주기), is_correct()(채점하기)입니다. 메서드와 run()의 관계는 부록 C의 보충 설명에 있습니다.

11) 파일 저장 (JSON)

- **뜻:** 프로그램을 끄면 메모리 속 내용은 사라집니다. 그래서 파일에 적어 두고 다음에 다시 읽습니다. JSON은 그 파일을 쓰는 형식으로, dict·list와 모양이 거의 같습니다.
- **비유:** 게임을 끄기 전에 공책에 적어 두고, 다음에 켜면 공책을 펼쳐 이어서 하는 것입니다.
- **과정:** 파일 열기 → 읽기("r") 또는 쓰기("w") → 닫기. with open(...)을 쓰면 닫기가 자동입니다.
- **우리 코드:** 퀴즈를 추가하거나 최고 점수가 바뀔 때마다 state.json에 저장합니다.

```
with open(STATE_FILE, "w", encoding="utf-8") as f:   # 공책을 쓰기용으로 펼치고
    json.dump(data, f, ensure_ascii=False, indent=4)  # 한글 그대로 적기
```

12) try / except

- **뜻:** 오류가 날 수 있는 코드를 try에 넣고, 오류가 나면 except에서 대신 처리합니다. 그러면 프로그램이 멈추지 않습니다.
- **비유:** 서커스의 안전망입니다. "일단 해 보고, 떨어지면 안전망이 받아 준다."
- **우리 코드:** 3곳에 안전망을 두었습니다. 숫자가 아닌 입력(ValueError), 망가진 저장 파일(JSONDecodeError), Ctrl+C(KeyboardInterrupt)입니다.

```
try:
    number = int(raw)     # "abc"를 숫자로 바꾸려면 오류
except ValueError:
    print("⚠ 잘못된 입력입니다...")   # 멈추지 않고 다시 묻기
```

13) Git / commit

- **뜻:** Git은 파일이 언제 어떻게 바뀌었는지 기록하는 도구입니다. commit은 지금 상태를 메시지와 함께 한 번 저장하는 것입니다. add는 어떤 파일을 저장할지 고르는 단계입니다.
- **비유:** 게임의 세이브 포인트입니다. 문제가 생기면 이전 세이브로 돌아갈 수 있고, add는 사진 찍기 전에 사진틀에 설 사람을 고르는 것입니다.
- **우리 작업:** 기능을 하나 완성할 때마다 커밋해 총 21번 저장했습니다.

```
git add .
git commit -m "Feat: 퀴즈 추가 기능 구현 (입력 검증 포함)"
```

14) push / pull

- **뜻:** push는 내 컴퓨터의 커밋을 GitHub에 올리는 것, pull은 GitHub의 새 내용을 내 컴퓨터로 내려받는 것입니다.
- **비유:** 휴대폰 사진을 클라우드에 올리기(push), 다른 기기에서 찍은 사진을 클라우드에서 받기(pull)입니다.
- **우리 작업:** 커밋할 때마다 push했고, 복제본에서 올린 변경을 원래 폴더에서 pull로 받았습니다.

15) branch / merge

- **뜻:** branch(브랜치)는 기록을 갈래로 나눠 따로 작업하는 것, merge(병합)는 그 갈래를 다시 합치는 것입니다. checkout은 갈래를 옮겨 다니는 명령입니다.
- **비유:** 공연 전 본무대(main)는 그대로 두고 연습실(branch)에서 새 곡을 연습한 뒤, 완성되면 본무대 프로그램에 넣는(merge) 것입니다.
- **왜 쓰나:** 작업 중에 문제가 생겨도 main의 잘 되는 코드는 안전하게 지킬 수 있습니다.
- **우리 작업:** 퀴즈 풀기(feature/play-quiz)와 랜덤 출제(feature/bonus)를 따로 만들고 main에 합쳤습니다.

```
git checkout -b feature/play-quiz   # 연습실 만들고 들어가기
git checkout main                   # 본무대로 돌아오기
git merge --no-ff feature/play-quiz # 연습한 것 합치기
```

16) clone

- **뜻:** GitHub에 있는 저장소를 기록까지 통째로 내 컴퓨터에 복사하는 것입니다.
- **비유:** 다른 컴퓨터에 프로젝트 폴더를 그대로 내려받는 것입니다. 단순 복사와 달리 지금까지의 세이브 기록도 함께 옵니다.
- **우리 작업:** quiz-clone 폴더에 복제해 README에 한 줄을 더하고 push한 뒤, 원래 폴더에서 pull로 받았습니다. 협업할 때 동료가 내 저장소를 받아 작업하는 방식과 같습니다.

```
git clone https://github.com/coolgirl2046/classical-music-quiz.git quiz-clone
```

부록 C. 예상 질문과 답변

과제 목표가 "스스로 설명할 수 있다"이므로, 목표 항목별로 나올 만한 질문과 이 프로젝트 코드를 예로 든 답변을 정리했습니다.

Python 기초

질문	답변
변수는 무엇이고 왜 쓰나요?	값에 이름을 붙여 담아 두는 상자입니다. 최고 점수를 best_score에 담아 두고 필요할 때 꺼내 비교합니다.
int, str, bool, list, dict의 차이는?	int는 정수(정답 번호, 점수), str은 글자(문제), bool은 참/거짓(정답 확인 결과), list는 순서 있는 여러 값(선택지 4개, 퀴즈 목록), dict는 이름표(키)와 값의 묶음(state.json 저장 형태)입니다.
if/elif/else는 어디에 썼나요?	run()에서 메뉴 번호에 따라 1이면 풀기, 2면 추가처럼 다른 기능을 실행합니다.
for와 while의 차이는?	for는 반복할 대상이 정해져 있을 때(퀴즈 목록을 하나씩 출제), while은 조건이 맞는 동안 계속 반복할 때(올바른 숫자를 입력할 때까지, 종료를 고를 때까지 메뉴 반복) 씁니다.
매개변수와 반환값의 예는?	read_number는 안내 문구와 최소·최대 범위를 매개변수로 받고, 검사를 통과한 숫자를 반환합니다. 메뉴(1~5)와 정답(1~4)에 같은 함수를 재사용합니다.

클래스와 객체

질문	답변
클래스는 무엇이고 왜 쓰나요?	객체를 만드는 설계도입니다. Quiz 클래스라는 틀로 문제 객체를 여러 개 만들었습니다. 관련된 데이터와 기능을 묶어 역할별로 정리할 수 있습니다.
__init__과 self의 역할은?	__init__은 객체가 만들어질 때 자동으로 실행되어 처음 값을 정합니다. self는 만들어진 그 객체 자신을 가리킵니다.
속성과 메서드의 예는?	속성은 객체가 가진 데이터(question, choices, answer), 메서드는 객체가 하는 동작(display, is_correct)입니다.
왜 클래스를 두 개로 나눴나요?	Quiz는 문제 하나, QuizGame은 게임 전체 진행을 담당합니다. 역할을 나눠 코드를 찾기 쉽고 고치기 쉽게 했습니다.

보충 설명: 메서드와 run()

메서드는 클래스 안에 들어 있는 "할 수 있는 일(기능)"입니다. `def 이름(self):`로 시작하는 덩어리 하나가 메서드 하나입니다. QuizGame에는 `play_quiz()`(퀴즈 내기), `add_quiz()`(추가하기), `show_quiz_list()`(목록 보기), `show_best_score()`(최고 점수 보기), `save_state()`(저장하기) 등이 있습니다.

`run()`은 그 메서드들 중 하나로, 게임 진행을 담당합니다. 메뉴판을 보여주고, 사용자가 고른 번호를 받아, 번호에 맞는 메서드를 부릅니다. 5번(종료)을 고를 때까지 이 과정을 반복합니다.

식당에 비유하면 메서드는 요리사가 만들 줄 아는 요리 하나하나이고, `run()`은 주문을 받아 주방에 전달하는 직원입니다. 새 요리를 배워도 직원이 그 주문을 받을 줄 모르면 손님에게 나가지 않습니다. 그래서 새 메서드를 만들면 `run()`에 연결해야 합니다.

파일 입출력

질문	답변
파일을 열고 읽고 쓰는 과정은?	파일을 열고, 읽거나 쓰고, 닫습니다. `with open(...)`을 쓰면 끝날 때 자동으로 닫힙니다. 한글이 깨지지 않도록 `encoding="utf-8"`을 지정했습니다.
JSON은 무엇이고 왜 썼나요?	데이터를 이름표와 값 형태의 글자로 저장하는 형식입니다. 사람도 읽기 쉽고 파이썬의 dict·list와 모양이 같아 변환이 쉽습니다.
try/except는 어디에 썼나요?	숫자가 아닌 입력(ValueError), 손상된 state.json(JSONDecodeError), Ctrl+C(KeyboardInterrupt)를 잡아 프로그램이 멈추지 않게 했습니다.
state.json은 왜 Git에 올리지 않았나요?	실행할 때마다 바뀌는 개인 데이터이고, 없으면 기본 퀴즈로 자동 생성되기 때문에 `.gitignore`로 제외했습니다.

Git 기초

질문	답변
Git은 무엇이고 왜 필요한가요?	파일의 변경 이력을 기록하는 도구입니다. 언제 무엇을 바꿨는지 남기고, 필요하면 되돌리거나 여러 사람이 함께 작업할 수 있습니다. GitHub는 그 기록을 올려 두는 온라인 저장소입니다.
명령어 7개는 각각 무엇을 하나요?	init은 저장소 만들기, add는 커밋할 파일 고르기, commit은 기록 남기기, push는 GitHub에 올리기, pull은 GitHub의 변경 내려받기, checkout은 브랜치 이동·생성, clone은 저장소 통째로 복제하기입니다.
브랜치는 왜 썼나요?	main을 건드리지 않고 새 기능을 따로 만들기 위해서입니다. feature/play-quiz와 feature/bonus에서 작업한 뒤 merge로 합쳤습니다.
clone과 pull은 어떻게 실습했나요?	저장소를 quiz-clone 폴더에 clone해 README에 한 줄을 추가하고 push했습니다. 원래 폴더에서 pull해 그 변경이 반영된 것을 확인했습니다.
커밋 메시지 오타는 왜 고치지 않았나요?	이미 GitHub에 올라간 기록을 고치려면 이력을 다시 써야 하고, 이는 협업에서 충돌을 일으킬 수 있어 그대로 두었습니다. push 전에는 `--amend`로 고칠 수 있다는 것도 배웠습니다.